SZY v8 — dokumentacja inżynieryjna silnika, formatu i testów Stratna kompresja danych naukowych 2D: transmisja, archiwizacja, szybki decode

Projekt: SzKompresor / SZY v8

1 maja 2026

Spis treści

1 Wstęp: kompresja stratna w nauce i integralność danych **3**
1.1 Kodek stratny vs bezstratny . . . 3
1.2 Silnik vs aplikacja (rola klienta) . . . 3
1.3 Dlaczego integralność jest częścią formatu . . . 3

2 Cel i zakres dokumentu (co jest "kompletne") **3**

3 SZY v8 jako silnik i format: zastosowania oraz nisze **4**
3.1 Czym jest SZY v8 (definicja inżynieryjna) . . . 4
3.2 Gdzie SZY v8 ma sens naukowo-techniczny . . . 4
3.3 Dlaczego porównania z liderami są konieczne nawet dla niszy . . . 6
3.4 Różnice względem SZ3 i ZFP (co porównujemy, a czego nie) . . . 6
3.5 Lossless jako baseline, nie cel . . . 7

4 Struktura projektu i moduły **7**
4.1 Opis modułów (pełny skrót) . . . 7
4.2 Kluczowe struktury biblioteki (C API) . . . 8
4.3 Wrapper Python: klasa `SzyDLL` . . . 9

5 Metryki i definicje **10**
5.1 Rozmiar, ratio, bpp . . . 10
5.2 Błędy . . . 10
5.3 Skala danych vs eps (ważne w benchmarkach) . . . 10

6 Specyfikacja przenośna: typy, endianness, varuint **10**
6.1 Endianness . . . 10
6.2 Varuint . . . 10

7 Format kontenera SZ2D v8 — pełna specyfikacja **10**
7.1 Layout kontenera (normatywnie) . . . 10
7.2 Znaczenie `flags` . . . 11
7.3 Index blob . . . 11
7.4 Integralność (SHA) i kody błędów . . . 11
7.5 Wymagania i walidacje (implementacja referencyjna) . . . 11

8 Format tile record v8 — pełna specyfikacja **11**
8.1 ID i flagi . . . 12
8.2 Layout tile record (normatywnie) . . . 12
8.3 Eventy: indeksy i amplitudy . . . 13
8.4 Payload i oczekiwany rozmiar po dekompresji . . . 13

9 Algorytm: encode/decode (szczegółowo) **13**
9.1 Encode: kroki na tile . . . 13
9.2 Decode: kroki na tile . . . 14

10 Równoległość: decode i encode po tile **14**
10.1 Decode parallel . . . 14
10.2 Encode parallel . . . 14

11 API silnika (DLL) i kody błędów **14**
11.1 Kody błędów . . . 14
11.2 Zasady pamięci (ABI) . . . 15
11.3 Polityka wątków . . . 15
11.4 Semantyka błędów API . . . 16
11.5 Wybrane eksporty DLL (silnik) . . . 16
11.6 Opis funkcji DLL (do czego służą i jak ich używać) . . . 16

12 Kompilacja (Windows/MSVC) **17**

13 Testy inżynieryjne i benchmarki: metodologia **18**
13.1 Identyfikacja artefaktu (SHA256 DLL) . . . 18
13.2 Dwa poziomy testów: auto-suite syntetyczny i benchmark publiczny . . . 18
13.3 Benchmark publiczny (bench_public.py): dane i kodeki . . . 18
13.4 Auto-suite v1: syntetyki + MNIST/Fashion-MNIST (starszy pipeline) . . . 19

14 Wyniki (manual): granula 10880×10880 float64 **19**
14.1 Porównanie z SZ3/ZFP i baseline lossless . . . 19

15 Wyniki (auto-suite v1): syntetyki + MNIST/Fashion-MNIST (starszy eksperyment) **19**
15.1 Wyniki na danych pobranych (MNIST/Fashion-MNIST): tabela zbiorcza . . . 20

16 Wyniki (public benchmark v3): dane satelitarne, sensory, syntetyczne **20**
16.1 Wybrane dataset-y i konfiguracje . . . 20
16.2 Interpretacja naukowo-techniczna wyników . . . 21

17 Najważniejsze wnioski (stan v8 po testach) **22**
17.1 Co działa i co jest wartością engine . . . 22
17.2 Czego porównania z SZ3/ZFP uczą (bez zmiany niszy) . . . 22
17.3 Rekomendacje rozwojowe (zgodne z filozofią tile-based engine) . . . 22
17.4 Potencjał naukowy i przewagi praktyczne . . . 23

A Załącznik A: Artefakty i reprodukowalność **24**

B Załącznik B: Minimalna specyfikacja zgodności (must-pass) **24**

Bibliografia **25**

1 Wstęp: kompresja stratna w nauce i integralność danych

1.1 Kodek stratny vs bezstratny

Kodek stratny (lossy) dopuszcza kontrolowane odchylenie danych wyjściowych od wejściowych w zamian za mniejszy rozmiar. W kontekście naukowym kluczowy jest **jawnie zdefiniowany model błędu**, a nie subiektywna jakość.

Typowe modele:

- **Bound L∞ (absolute error bound)**: $\max_i |x_i - \hat{x}_i| \leq \varepsilon$.
- **Bound względny**: $\max_i \frac{|x_i - \hat{x}_i|}{|x_i| + \delta} \leq \varepsilon_{rel}$.
- **Miary energetyczne**: RMSE, L2, PSNR jako dodatkowa charakterystyka.

Uwaga o "zaokrągleniu po 12. miejscu po przecinku". W wielu zastosowaniach naukowych błąd rzędu 10^{-12} (w skali danych ≈ 1) jest akceptowalny. Należy jednak pamiętać, że sensowność takiej tolerancji zależy od *skali* i jednostek danych: dla wartości rzędu 10^6 lub 10^7 precyzja reprezentacji `float64` (ULP) może być większa niż 10^{-12}.

1.2 Silnik vs aplikacja (rola klienta)

SZY v8 jest **silnikiem (engine)**. Oznacza to, że:

- **silnik dostarcza mechanikę** (format, pipeline, integralność, wątki, decode-into),
- **klient dobiera parametry** (np. `eps`, blok `B`, liczba workerów, tryb kwantyzacji, politykę eventów),
- poprawność "naukowa" jest oceniana w kontekście *realnych danych* i wymagań klienta,
- dla klas danych *niedopasowanych do modelu* silnik może jawnie odmówić spełnienia zadanego bounded-error zamiast generować wynik łamiący gwarancję.

Testy w niniejszym dokumencie pokazują, że przy sensownie dobranych parametrach dla klas danych obrazopodobnych i szumowych, SZY spełnia oczekiwania jakościowe (kontrolowany błąd) oraz zapewnia korzystne trade-offy względem referencyjnych kodeków. Jednocześnie benchmark zawiera również przypadki *stress-test / out-of-domain*, których celem nie jest dowodzenie przewagi modelu, lecz wyznaczenie granic jego stosowalności.

1.3 Dlaczego integralność jest częścią formatu

W praktyce występuje ryzyko **silent data corruption**. W silniku SZY v8 integralność jest częścią kontenera:

- `data_sha`: SHA-256 sekcji danych (indeks + tile records),
- `container_sha`: SHA-256 całego kontenera.

Pozwala to wykrywać uszkodzenia i jednoznacznie zwrócić błędy: `SZY_E_SHA_DATA` i `SZY_E_SHA_CONT`.

2 Cel i zakres dokumentu (co jest "kompletne")

Dokument obejmuje:

1. **Specyfikację binarną** formatu kontenera SZ2D v8 oraz tile record v8 (layout, typy, varuint, endianness, SHA).

2. **Opis algorytmu** encode/decode (pipeline transformacji i ich kolejność).
3. **Opis silnika (engine)**: API DLL/ABI, kontrola wątków, decode-into, indeks, integralność.
4. **Kompilację** (MSVC) i zależności (zstd/zlib).
5. **Testy inżynieryjne** (manual, public benchmark): metodologia, dataset-y, wyniki, obserwacje.
6. **Pozycjonowanie względem liderów** (SZ3 i ZFP): różnice filozofii, sens porównań, wnioski.

Uwaga: `eps=0` nie oznacza lossless. W wersji v8 `eps=0` wyłącza model bounded-error, ale **nie** wyłącza kwantyzacji (wciąż jest to kompresja stratna). Tryb lossless nie jest celem v8.

3 SZY v8 jako silnik i format: zastosowania oraz nisze

3.1 Czym jest SZY v8 (definicja inżynieryjna)

SZY v8 jest **tile-based bounded-error codec** dla danych 2D oraz **kontenerem** z integralnością i indeksem, udostępnionym jako **biblioteka C / DLL**.

Najważniejsze cechy silnika:

- **Tile independence**: obraz dzielony na kafelki $B \times B$, każdy tile record jest niezależny. To pozwala na:
  - lokalną kompresję/dekompresję (region-of-interest),
  - łatwą równoległość po tile,
  - odporność na lokalne uszkodzenia (uszkodzenie jednego tile nie unieważnia całości).
- **Index blob**: lista długości tile (varuint), umożliwia szybkie wyznaczenie offsetów tile i równomierny podział pracy między wątkami.
- **Równoległy decode**: dekodowanie tile równolegle na podstawie indeksu, z kontrolą liczby workerów i progu `parallel_min_tiles`.
- **Decode-into**: dekompresja bezpośrednio do bufora użytkownika (bez dodatkowej alokacji obrazu) — istotne dla kodów HPC/FORTRAN, które same zarządzają pamięcią.
- **Integralność**: SHA-256 danych i kontenera jako część formatu; użytkownik nie musi implementować własnych sum kontrolnych.
- **Obsługa typów**: float64, float32, uint16 — pokrywa typowe dane naukowe (pola klimatyczne, sensory, intensywności obrazów).
- **Wieloplatformowość**: little-endian format, implementacja w C, możliwość uruchomienia na x86-64 i ARM (w tym edge/embedded).

3.2 Gdzie SZY v8 ma sens naukowo-techniczny

Silnik jest szczególnie przydatny w następujących scenariuszach:

Klimat i oceanografia (NetCDF, siatki geofizyczne).

- Pola 2D (SST, temperatura przy powierzchni, ciśnienie) z modeli klimatycznych lub reanaliz,
- Horyzontalne przekroje z symulacji 3D (ROMS, NEMO, ICON),
- Strumienie danych z serwisów katalogowych (THREDDS, pyDAP) po konwersji do SUROWYCH bloków C-ordered.

Dla takich danych kluczowe są:

- zachowanie kształtu pola (gradienty, struktury mezoskalowe),
- kontrola błędu (aby nie zepsuć asymilacji i diagnostyk wrażliwych),
- przepustowość (HBM/SSD/ethernet).

Benchmark pokazuje, że na syntetycznym polu klimatycznym (f32) SZY osiąga ratio 10^3 przy eps=10–20 i spełnia bound.

Obrazy satelitarne i sensory optyczne.

- GeoTIFF/CCSDS wyciągnięte jako uint16,
- czujniki optyczne (Landsat, Sentinel) po redukcji do pojedynczej bandy,
- downlink i archiwizacja w stacjach naziemnych.

Dla takiej klasy danych, SZY v8:

- daje ratio ∼8–16x przy eps=5–20, podczas gdy Zstd/Blosc2 ∼4.4x,
- zapewnia twardy bound na błędzie (istotne dla dalszego przetwarzania, np. indeksy wegetacji),
- pozwala na decode fragmentów obrazu (tile).

Dane medyczne (CT) i astronomiczne (CCD).

- Przykładowe syntetyczne CT/CCD w formacie uint16 (symulujące HU lub ADU),
- Zstd/Blosc2 oferują ∼1.7–1.8x, SZY przy eps=20 osiąga 5–6x (z kontrolą błędu).

SZY jest naturalnym kandydatem dla archiwizacji długoterminowej: można dobrać eps tak, aby błąd był istotnie mniejszy od wariancji szumu/aparatury.

Strumienie sensoryczne (UCI: AirQuality, Energy) i szeregi czasowe.

- Dla `sensor_energy_uci` przy eps=20, B=128: SZY daje 532x przy max_err≈19.6; SZ3 daje 470x, ZFP ∼126x, Zstd ∼38x.
- Dla `sensor_airquality_uci` przy eps=20, B=128: SZY i SZ3 są w tej samej klasie ratio (196x vs 206x), SZY jest szybszy.

To sugeruje potencjał SZY jako "klocka" w pipeline'ach IoT/telemetria, gdzie:

- strumienie są większe niż możliwość trzymania raw,
- dopuszcza się mały, jawny błąd pomiaru,
- istotna jest szybkość enkodera/decodera na CPU klasy laptop/edge.

Symulacje HPC (CFD, turbulencja, zjawiska mezoskalowe).

- Cross-sectiony 2D z plików HDF5/NetCDF (np. Hurricane, Miranda),
- zapisy checkpointów do archiwum (tile-encoded),
- odczyt częściowy przez decode-into (MPI rank odczytuje tylko swój fragment).

SZY v8 dostarcza tutaj nie tylko ratio, ale i engine: łatwość integracji z istniejącymi kodami Fortrana/C++ (biblioteka C, decode-into buffer).

3.3 Dlaczego porównania z liderami są konieczne nawet dla niszy

Jeżeli buduje się niszę, nadal trzeba porównywać do liderów, bo:

- rynek rozumie SZ3 i ZFP jako punkty odniesienia (kod naukowy vs "kodek do obrazków"),
- benchmark ustala mapę trade-off: bitrate vs błąd vs czas,
- porównania ujawniają klasy danych, gdzie warto dodać opcjonalne tryby (np. trend, inne predyktory).

Uwaga o benchmarkach out-of-domain. W testach pojawiają się również dane syntetyczne o strukturze skrajnie niekorzystnej dla aktualnego modelu (np. ostre nieciągłości na granicach tile). Nie są to dane docelowe, ale są użyteczne jako **benchmark graniczny**: pokazują, kiedy metoda traci efektywność albo powinna jawnie zgłosić błąd bounded-error zamiast udawać sukces.

3.4 Różnice względem SZ3 i ZFP (co porównujemy, a czego nie)

ZFP (rozwiązanie wyspecjalizowane). ZFP [2] jest wyspecjalizowany w kompresji tablic FP (blokowo, z trybami fixed-rate/fixed-precision/fixed-accuracy). Jest świetnym punktem odniesienia jako *czysty kodek liczb FP*, ale:

- nie jest kontenerem-engine (brak SHA, indexu, decode-into w standardzie),
- nie obsługuje uint16 jako typu pierwszej klasy (wymagana konwersja).

W benchmarkach SZY v8 bardzo często **bije ZFP przy tym samym eps** pod względem ratio, szczególnie na danych sensorowych i obrazopodobnych.

SZ3 (framework / często maksymalizuje bitrate). SZ3 [3] jest modularnym frameworkiem, często osiągającym bardzo wysokie ratio na klasach danych pasujących do jego strategii (zwłaszcza bardzo gładkie pola). Jest dobrym odniesieniem jako "górna granica" bitrate w zadanym modelu błędu. Z drugiej strony:

- jest złożony w konfiguracji i integracji (różne tryby, pipeline),
- nie ma wbudowanego kontenera z SHA i indexem kafelków,
- bywa wolniejszy niż SZY v8 na tych samych datasetach przy tym samym eps.

SZY v8 (pozycja). SZY v8 optymalizuje jednocześnie:

- bounded $L\infty$ (kontrolowany błąd),
- prędkość encode/decode i MT decode,
- integralność i odporność (SHA),
- ergonomię integracji (decode-into, peek-shape, C API, wrapper Py),
- wsparcie uint16 (istotne dla obrazów).

Z benchmarków:

- na gładkich polach klimatycznych SZ3 ma lepszy bitrate (2–3×),
- na realnych danych sensorowych SZY potrafi przebić SZ3 w ratio przy tym samym eps (np. `sensor_energy_uci`),
- SZY konsekwentnie bije ZFP przy tym samym eps i jest znacznie lepszy niż lossless baseline dla większości datasetów.

3.5 Lossless jako baseline, nie cel

W testach pojawiają się kodeki bezstratne (Zstd, Blosc2) jako **baseline**:

- pokazują, ile redundancji jest w danych bez strat,
- pozwalają ocenić, czy w ogóle opłaca się lossy dla danej klasy danych,
- nie stanowią celu projektowego SZY v8 (SZY jest projektowany pod lossy scientific).

4 Struktura projektu i moduły

Przykładowy układ katalogów:

```
src/
  szy_bitplane.h, szy_bitplane.c
  szy_buf.h,  szy_buf.c
  szy_bytes.h, szy_bytes.c
  szy_endian.h
  szy_varuint.h, szy_varuint.c
  szy_sha256.h, szy_sha256.c
  szy_container_v8.h, szy_container_v8.c
  szy_tile_v8.h, szy_tile_v8.c
  szy_parallel.h, szy_parallel.c
  szy_decode_v8.h, szy_decode_v8.c
  szy_encode_v8.h, szy_encode_v8.c
  szy_encode_f32.c
  szy_encode_u16.c
  szy_hw.h, szy_hw.c
  szy_rc.h
  szy_errors.c
  szy_exports.c
include/
  szy_api.h
  szy_dll.h
```

4.1 Opis modułów (pełny skrót)

Uwaga o zapisie `.[ch]`. W dokumentacji stosowany jest skrót `foo.[ch]` oznaczający **parę plików** `foo.c` (implementacja) oraz `foo.h` (deklaracje). Jest to konwencja techniczna, a nie rzeczywiste rozszerzenie pliku.

- `szy_bitplane.[ch]`: pomocniczy moduł transformacji bitplanów 16-bitowych (u16) — aktualna wersja implementuje `szy_bitplane_shuffle_u16()` i `szy_bitplane_unshuffle_u16()`, gotowe do użycia w pipeline (C-encoder v8 domyślnie używa shuffle16, ale bitplane jest w pełni zaimplementowane i może zostać włączone w kolejnych wersjach API).
- `szy_buf.[ch]`: writer rosnącego bufora, zapisy typów LE.
- `szy_bytes.[ch]`: reader/cursor do bezpiecznego parsowania (kontrola zakresów).
- `szy_endian.h`: przenośne store/load dla u16/u32/u64 oraz f32/f64 w LE.
- `szy_varuint.[ch]`: varuint decode/encode (7-bit payload + MSB continuation).
- `szy_sha256.[ch]`: implementacja SHA-256 (bez zależności zewnętrznych).

- `szy_container_v8.[ch]`: parse kontenera v8, weryfikacja SHA, odczyt indeksu długości tile.
- `szy_tile_v8.[ch]`: dekodowanie pojedynczego tile record (entropia + undo transformacji + predyktor).
- `szy_parallel.[ch]`: równoległy decode tile na podstawie indeksu (Windows/POSIX).
- `szy_decode_v8.[ch]`: decode całego obrazu (alokacja wyjścia) oraz logika wyboru trybu parallel/sequential.
- `szy_encode_v8.[ch]`: encode całego obrazu, budowa kontenera, eventy, entropia (ZSTD/ZLIB/NONE), transformacje (Lorenzo, bias, zigzag, shuffle16/bitplane).
- `szy_encode_f32.c, szy_encode_u16.c`: wrappery typów (rzucony na f64 pipeline).
- `szy_hw.[ch]`: `szy_hw_threads()` + walidacja `num_workers` (strict/relaxed).
- `szy_exports.c`: eksporty DLL, API `*_ex` (wątki na zewnątrz).
- `szy_errors.c`: `szy_strerror()`.

4.2 Kluczowe struktury biblioteki (C API)

Dla klienta C/C++ istotne są następujące typy:

`szy_enc_cfg_t.` Struktura konfiguracji encodera:

```c
typedef struct {
  int block;                /* np. 64/128 */
  int qbits;                /* 8 lub 16 */
  szy_quant_mode_t qmode;   /* strict/robust */
  double qpercentile;       /* np. 99.5 przy robust */

  /* bounded error model */
  double target_abs_err;    /* eps; <=0 => bounded-error off */
  double bounded_safety;    /* np. 0.98 */
  double quant_err_factor;  /* np. 2.0 */

  /* events */
  int events_on;            /* 0/1 */
  int events_max_per_tile;  /* np. 2048 */
  double events_max_frac;   /* np. 0.01 */

  /* entropy */
  szy_entropy_t ent;        /* ZSTD/ZLIB/NONE */
  int zstd_level;           /* 1..22 */

  /* transforms */
  int use_zigzag;
  int byte_shuffle_16;
  int use_bitplane_shuffle; /* infrastruktura gotowa, domylnie 0 */

  /* weryfikacja post-factum (opcjonalnie) */
  int verify_after_encode;  /* 0/1 */
} szy_enc_cfg_t;
```

Typowa konfiguracja używana przez DLL ustawia:

- `qbits=16`, `qmode=strict`, `target_abs_err=eps`,
- `events_on=1`, odpowiedni budżet eventów,

- entropię ZSTD (lvl=3),
- zigzag + shuffle16 (bitplane off w v8 DLL).

szy_container_hdr_t. Struktura nagłówka kontenera po szy_container_unpack_v8():

```c
typedef struct {
  uint16_t H, W, block;
  uint8_t version;
  uint8_t flags;

  int index_present;
  uint32_t ntiles;
  uint32_t* tile_lengths; /* optional; malloc'd */

  size_t data_start;
  size_t tiles_start;
  size_t data_end;
} szy_container_hdr_t;
```

Używana wewnętrznie przez dekoder i w szy_peek_shape_v8.

szy_bw_t (writer) i szy_rd_t (reader). Proste struktury do robienia zapisu/odczytu binarnego z kontrolą zakresów i łatwym zarządzaniem buforem:

```c
typedef struct {
  uint8_t* p;
  size_t n;
  size_t cap;
} szy_bw_t;

typedef struct {
  const uint8_t* p;
  size_t n;
  size_t pos;
} szy_rd_t;
```

4.3 Wrapper Python: klasa SzyDLL

Do benchmarków używany jest prosty wrapper ctypes:

Listing 1: Klasa SzyDLL (Python, bench$_{c}$*ommon.py*)

```python
class SzyDLL:
    def __init__(self, dll_path: str):
        self.dll_path = os.path.abspath(dll_path)
        self.lib = ctypes.CDLL(self.dll_path)
        # mapowanie funkcji DLL na sygnatury ctypes (...)
    def compress(self, x: np.ndarray, eps: float, block: int, num_workers: int) -> bytes:
        # wybr szy_compress_*_ex w zalenoci od dtype (f64/f32/u16)
        # zwrot bajtw kontenera
    def decompress_into_f64(self, blob: bytes, out: np.ndarray,
                            num_workers: int, parallel_min_tiles: int) -> (int, int):
        # wywoanie szy_decompress_into_buffer_ex2
```

Dzięki temu benchmarki w Pythonie widzą SZY jako "normalny" kodek (tak jak zfpy, pysz, zstandard), a cały engine (SHA, index, decode-into) pozostaje w C.

5 Metryki i definicje

5.1 Rozmiar, ratio, bpp

- **ratio** $= \frac{\text{raw_bytes}}{\text{compressed_bytes}}$
- **bpp** (bits-per-pixel) $= \frac{8 \cdot \text{compressed_bytes}}{H \cdot W}$

5.2 Błędy

- **max_err** $= \max_i |x_i - \hat{x}_i|$
- **RMSE** $= \sqrt{\frac{1}{N} \sum_i (x_i - \hat{x}_i)^2}$
- **p99_err** $= 99$ percentyl $|x_i - \hat{x}_i|$

5.3 Skala danych vs eps (ważne w benchmarkach)

Auto-suite i benchmark publiczny raportują p01/p99/std. W praktyce:

$$\texttt{eps_over_range} = \frac{\varepsilon}{p99 - p01}$$

umożliwia porównywanie „agresywności eps” między datasetami o różnych skalach.

6 Specyfikacja przenośna: typy, endianness, varuint

6.1 Endianness

W kontenerze i tile record liczby stałej długości są kodowane w **little-endian**: `u8, u16le, u32le, u64le, i16le, f32le, f64le`. Floaty są IEEE-754.

6.2 Varuint

Varuint koduje `uint64` jako sekwencję bajtów:

- każdy bajt przenosi 7 bitów wartości,
- bit MSB=1 oznacza kontynuację,
- bit MSB=0 kończy liczbę.

7 Format kontenera SZ2D v8 — pełna specyfikacja

7.1 Layout kontenera (normatywnie)

Tabela 1: Kontener SZ2D v8: layout binarny (kolejność pól).

Pole	Typ	Rozmiar	Opis
magic	bytes	4	ASCII `"SZ2D"`
version	u8	1	wartość `8`
flags	u8	1	bit0: `index_present`
H	u16le	2	wysokość obrazu
W	u16le	2	szerokość obrazu
B	u16le	2	rozmiar tile (blok)

Pole	Typ	Rozmiar	Opis
meta_len	u16le	2	długość metadanych
meta	bytes	meta_len	blob (np. JSON)
data_sha	bytes	32	SHA256(`data_section`)
data_section	bytes	var	`index_blob || tiles_blob`
container_sha	bytes	32	SHA256(kontener bez tego pola)

7.2 Znaczenie `flags`

- `flags & 1` = 1: indeks tile jest obecny na początku `data_section`.
- `flags & 1` = 0: brak indeksu; tile records idą sekwencyjnie.

7.3 Index blob

Jeśli indeks jest obecny, `data_section` zaczyna się od:

- `ntiles` (varuint),
- `tile_len[i]` (varuint) dla i=0..ntiles-1,
- następnie `tiles_blob`: sklejone tile recordy.

7.4 Integralność (SHA) i kody błędów

- `SZY_E_SHA_CONT`: mismatch SHA kontenera,
- `SZY_E_SHA_DATA`: mismatch SHA sekcji danych.

7.5 Wymagania i walidacje (implementacja referencyjna)

Parser w C weryfikuje m.in.:

- `magic="SZ2D"` i `version=8`,
- `H,W,B>0`,
- poprawne granice sekcji danych,
- spójność indeksu (sumy długości tile),
- zgodność SHA.

8 Format tile record v8 — pełna specyfikacja

Uwaga: specyfikacja formatu vs bieżąca implementacja C-encodera. Specyfikacja tile record v8 obejmuje pola i tryby, które dekoder referencyjny rozumie z powodów kompatybilności i rozszerzalności. Nie oznacza to, że bieżący C-encoder generuje wszystkie warianty. W szczególności aktualny C-encoder:

- używa predyktora Lorenzo2D,
- nie generuje trendów innych niż `TREND_NONE`,
- posiada infrastrukturę shuffle16 *i* bitplane shuffle dla qbits=16, ale DLL v8 domyślnie używa shuffle16; bitplane może być włączone w kompilacjach eksperymentalnych,
- nie generuje już `amp_mode=2` (float16) dla nowych strumieni (dekoder nadal akceptuje tryb 2 wyłącznie dla zgodności wstecz).

8.1 ID i flagi

Entropy ID (`ent_id`).

- 0: ZLIB
- 2: NONE
- 3: ZSTD

Predictor ID (`pred_id`).

- 0: Lorenzo2D (używane przez C-encoder v8)
- (dekoder wspiera także DIFFX/DIFFY jako kompatybilność)

Tile flags (`flags`).

- bit0 (1): SHUFFLE16
- bit1 (2): ZIGZAG
- bit2 (4): BITPLANE (dekoder wspiera; C-encoder v8 DLL domyślnie nie używa, ale kod shuffle/unshuffle jest włączony)

8.2 Layout tile record (normatywnie)

Tabela 2: Tile record v8: layout binarny (kolejność pól).

Pole	Typ	Rozmiar	Opis
th	u16le	2	wysokość tile (powinna równać się B)
tw	u16le	2	szerokość tile (powinna równać się B)
trend_id	u8	1	0=none (C-encoder v8)
qbits	u8	1	8 lub 16
qmode	u8	1	0=strict, 1=robust (dekoder ignoruje)
events_id	u8	1	0=off, 1=on
ent_id	u8	1	0=zlib,2=none,3=zstd
flags	u8	1	shuffle/zigzag/bitplane
pred_id	u8	1	0=Lorenzo2D
trend_params_len	u8	1	0/3/6 (liczba f64)
trend_params	f64le[]	8*len	parametry trendu
inv_scale	f64le	8	skala kwantyzacji
mean	f64le	8	średnia tile
clip_rate	f32le	4	statystyka informacyjna
bias	i16le	2	bias medianowy delt
has_pred_params	u8	1	jeśli 1: 16B reserved
pred_params	bytes	16	reserved
ne	varuint	var	liczba eventów
idx_len	varuint	var	długość idx_bytes
idx_bytes	bytes	idx_len	indeksy (delta-coded varuint)
amp_mode	u8	1	format amplitud
amp_blob	bytes	var	amplitudy wg trybu
payload_len	varuint	var	długość payload
payload	bytes	payload_len	dane skompresowane entropijnie

8.3 Eventy: indeksy i amplitudy

Indeksy. Kodowane jako varuint różnic (delta coding) po indeksach w spłaszczonym tile (row-major).

Amplitudy. `amp_mode` definiuje reprezentację amplitud eventów:

- 0: scale (f64le) + `ne` bajtów int8,
- 1: scale (f64le) + `ne` wartości int16le,
- 2: `ne` wartości float16 (u16le) — tryb legacy, tylko do odczytu,
- 3: `ne` wartości float32le.

Stan aktualny C-encodera v8. Aktualna implementacja C-encodera **nie generuje już trybu 2 (float16)** dla nowych strumieni, ponieważ dla dużych amplitud mógł on prowadzić do wartości niefinitych i niestabilnej rekonstrukcji. Dekoder nadal akceptuje `amp_mode=2` wyłącznie w celu **zgodności wstecz** z wcześniejszymi plikami.

Znaczenie eventów dla bounded-error. Eventy reprezentują lokalne odchylenia, których nie da się efektywnie obsłużyć samą kwantyzacją i predykcją przy zadanym `eps`. Jeśli liczba wymaganych eventów przekracza budżet narzucony konfiguracją klienta, silnik nie powinien „udawać sukcesu"; powinien zwrócić błąd bounded-error (por. sekcja o kodach błędów).

8.4 Payload i oczekiwany rozmiar po dekompresji

Po dekompresji entropijnej payload musi mieć rozmiar:

$$\texttt{expected} = \texttt{npix} \cdot \frac{\texttt{qbits}}{8}, \quad \texttt{npix} = \texttt{th} \cdot \texttt{tw}.$$

9 Algorytm: encode/decode (szczegółowo)

9.1 Encode: kroki na tile

Dla tile $B \times B$ (npix=B^2):

1. Oblicz średnią μ i wycentruj tile: $c_i = x_i - \mu$.
2. (Opcjonalnie) eventy: wytnij rzadkie odchylenia $|c_i|$ powyżej progu do listy (idx+amp), o ile mieszczą się w budżecie eventów narzuconym konfiguracją.
3. Ustal `inv_scale` dla kwantyzacji (zależnie od trybu i eps).
4. Kwantyzuj do int8/int16: $q_i = \text{round}(c_i \cdot inv_scale)$, z clip.
5. Predykcja Lorenzo2D na $q \to$ delty d (mod 2^{qbits}).
6. Bias medianowy delt: $d'_i = d_i - bias$ (bias liczymy jako medianę signed).
7. (Opcjonalnie) zigzag: signed $\to$ unsigned.
8. (Dla qbits=16) shuffle16 lub bitplane shuffle (jeśli włączony w konfiguracji).
9. Entropy coding: ZSTD/ZLIB/NONE.
10. Zapis tile record: nagłówek + parametry + eventy + payload.

Ważna własność aktualnej implementacji. Jeżeli w bounded-error mode liczba wymaganych eventów przekracza ograniczenia (`events_max_per_tile`, `events_max_frac`), encoder **zwraca** `SZY_E_BOUNDS`. Dodatkowo wprowadzono mechanizm wewnętrznego fallbacku per-tile (np. rozluźnienie eps, wyłączenie eventów) w implementacji, ale z punktu widzenia kontraktu zewnętrznego **bounded-error ma pierwszeństwo**: jeśli nie da się go utrzymać, lepiej zwrócić błąd niż złamać warunek.

9.2 Decode: kroki na tile

1. Odczyt tile record.
2. Dekompress payload (ZSTD/ZLIB/NONE) → body o rozmiarze `expected`.
3. Undo shuffle16 lub bitplane (jeśli flagi).
4. Undo zigzag (jeśli flaga).
5. Bias: $d_i = d'_i + bias$.
6. Predyktor inverse (Lorenzo2D) → q_i.
7. Dekwantyzacja: $c_i \approx q_i / inv_scale$.
8. Odtworzenie wartości: $\hat{x}_i = c_i + \mu + trend(i, j)$.
9. Dodanie eventów (jeśli obecne): $\hat{x}_{idx_k} \mathrel{+}= amp_k$.

10 Równoległość: decode i encode po tile

10.1 Decode parallel

Jeśli indeks jest obecny, silnik oblicza offsety tile na podstawie długości i uruchamia workerów. W implementacji referencyjnej:

- POSIX: prosty thread-pool (pthreads),
- Windows: pula workerów (stała liczba), nie "thread per tile".

10.2 Encode parallel

Encode może być wykonywany równolegle po tile, a następnie tile records są sklejane w kolejności deterministycznej (ti=0..ntiles-1) w celu utrzymania powtarzalnego kontenera.

11 API silnika (DLL) i kody błędów

11.1 Kody błędów

Listing 2: Kody błędów (szy_rc.h)

```
typedef enum {
  SZY_OK           = 0,
  SZY_EINVAL       = -1,
  SZY_E_SHA_CONT   = -2,
  SZY_E_SHA_DATA   = -3,
  SZY_E_INDEX      = -4,
  SZY_E_DIMS       = -5,
  SZY_E_BLOCK      = -6,
  SZY_E_TILECOUNT  = -7,
```

```
  SZY_E_OVERFLOW   = -8,
  SZY_E_OOM        = -9,
  SZY_E_TILE       = -10,
  SZY_E_THREADS    = -11,
  SZY_E_BOUNDS     = -12
} szy_rc_t;
```

Opis kodów błędów (znaczenie praktyczne). Same nazwy i wartości liczbowe nie są wystarczające w dokumentacji użytkowej; poniższa tabela opisuje, *co dany błąd oznacza* i *w jakich sytuacjach typowo się pojawia.*

Tabela 3: Znaczenie kodów błędów SZY v8 (kontrakt API).

Kod	Symbol	Klasa	Znaczenie / typowe przyczyny
0	`SZY_OK`	OK	Sukces. Operacja zakończona poprawnie.
-1	`SZY_E_INVAL`	Input/Format	Niepoprawny argument lub uszkodzony/niezgodny format (np. złe magic/version, niepoprawne pola tile, błędne długości, niepoprawny payload).
-2	`SZY_E_SHA_CONT`	Integrity	Mismatch SHA256 kontenera (korupcja kontenera lub niezgodność danych).
-3	`SZY_E_SHA_DATA`	Integrity	Mismatch SHA256 sekcji danych (index+tile blob).
-4	`SZY_E_INDEX`	Format/Index	Indeks tile jest niespójny (sumy długości, offsety wychodzą poza `data_end`, korupcja index blob).
-5	`SZY_E_DIMS`	Input	Nieprawidłowe wymiary `H/W/B` (np. 0 lub przekroczenie limitów).
-6	`SZY_E_BLOCK`	Input	Wymiary niepodzielne przez blok `B` (w decode) lub inny błąd warunku blokowego.
-7	`SZY_E_TILECOUNT`	Format	Niezgodna liczba kafli (`ntiles`) względem `H/W/B`.
-8	`SZY_E_OVERFLOW`	Safety	Przepełnienie rozmiaru (np. w obliczeniach offsetów/alokacji).
-9	`SZY_E_OOM`	Resources	Brak pamięci (alokacja nie powiodła się).
-10	`SZY_E_TILE`	Decode	Błąd dekodowania kafla (np. dekompresja entropijna niezgodna z oczekiwaną długością, nieobsługiwany wariant, niespójne pola tile).
-11	`SZY_E_THREADS`	Config	Żądana liczba wątków przekracza dostępne wątki sprzętowe (w trybie strict).
-12	`SZY_E_BOUNDS`	Quality/Bound	Kontrolowana odmowa kompresji: przy bieżącej konfiguracji bounded-error (np. `eps`, blok, budżet eventów) silnik nie może zagwarantować `max_err` w zadanym modelu. To **nie** jest korupcja danych.

11.2 Zasady pamięci (ABI)

- Bufor wyjściowy z `compress` jest alokowany w DLL i musi być zwolniony przez `szy_free_buffer()`.
- `decode-into` zapisuje do bufora użytkownika bez dodatkowych kopii (wymaga bufora odpowiedniego rozmiaru).

11.3 Polityka wątków

- `szy_hw_threads()` zwraca liczbę dostępnych wątków sprzętowych.
- API `*_ex` przyjmuje `num_workers`:

  - 0 = auto,
  - > 0 = wymuszone.

- Gdy żądana liczba wątków przekracza dostępne: `SZY_E_THREADS`.

11.4 Semantyka błędów API

Nie wszystkie błędy zwracane przez API oznaczają uszkodzony plik lub błąd implementacji. W praktyce:

- `SZY_E_SHA_*`, `SZY_E_INDEX`, `SZY_E_TILE` wskazują na problem integralności lub poprawności strumienia,
- `SZY_E_THREADS` wskazuje na błędną konfigurację wykonania,
- `SZY_E_BOUNDS` wskazuje na sytuację, w której przy bieżących ustawieniach modelu nie da się spełnić bounded-error dla danych wejściowych.

Z punktu widzenia systemowego `SZY_E_BOUNDS` jest zachowaniem pożądanym: lepiej jawnie odmówić kompresji niż zwrócić strumień łamiący deklarowane ograniczenie błędu.

11.5 Wybrane eksporty DLL (silnik)

Listing 3: Wybrane funkcje DLL (include/szy_dll.h)

```
SZY_API int szy_hw_threads(void);
SZY_API void szy_free_buffer(void* ptr);
SZY_API const char* szy_strerror(int code);

SZY_API int szy_peek_shape_v8(const uint8_t* buf, size_t n,
                              int* H, int* W, int* B);

/* decode into */
SZY_API int szy_decompress_into_buffer_ex2(
    const uint8_t* buf, size_t n,
    double* out, size_t out_elems,
    int num_workers, int parallel_min_tiles,
    int* H, int* W);

/* threaded compress (float64/f32/u16) */
SZY_API int szy_compress_buffer_ex(
    double* x, int H, int W, double eps, int B, int num_workers,
    uint8_t** out_buf, size_t* out_n);

SZY_API int szy_compress_f32_ex(...);
SZY_API int szy_compress_u16_ex(...);
```

11.6 Opis funkcji DLL (do czego służą i jak ich używać)

Poniższy opis odnosi się do Listingu powyżej (wybrane eksporty). Celem jest jednoznaczne wyjaśnienie semantyki parametrów, pamięci i kodów zwrotnych.

`szy_hw_threads()`. Zwraca liczbę dostępnych wątków sprzętowych (co najmniej 1). Funkcja jest przydatna do ustawienia `num_workers=0` (auto) lub do walidacji, czy aplikacja nie żąda zbyt wielu wątków.

`szy_free_buffer(ptr).` Zwalnia bufor zaalokowany przez DLL (np. wynik `compress`). **Kontrakt:** ten wskaźnik musi pochodzić z SZY; nie wolno go zwalniać inną metodą (np. `free()` w aplikacji), aby nie mieszać allocatorów.

`szy_strerror(code).` Zwraca stały napis opisujący kod błędu. Pomocne w logach i diagnostyce.

`szy_peek_shape_v8(buf,n,H,W,B).` Odczytuje tylko nagłówek kontenera i zwraca wymiary (H,W) oraz blok (B) **bez dekodowania danych**. Funkcja weryfikuje integralność kontenera (SHA) tak jak standardowy unpack.

`szy_decompress_into_buffer_ex2(...)` **(decode-into).** Dekoduje kontener do bufora użytkownika:

- `out` wskazuje na tablicę `double` o rozmiarze co najmniej `H*W` elementów,
- `out_elems` to liczba elementów w `out` (nie bajtów),
- `num_workers`: 0=auto, >0=żądana liczba wątków (tryb strict),
- `parallel_min_tiles` określa od ilu tile opłaca się przejść w tryb równoległy.

W przypadku błędu dekodera zawartość `out` należy uznać za nieważną.

`szy_compress_buffer_ex(...), szy_compress_f32_ex(...), szy_compress_u16_ex(...).` Kodują odpowiednio macierz `double`, `float` lub `uint16` do bufora kontenera SZ2D v8.

- `eps` jest parametrem modelu bounded-error (absolutny bound),
- `B` to rozmiar tile (np. 64 lub 128),
- `num_workers`: 0=auto, >0=żądana liczba wątków (tryb strict),
- `out_buf/out_n` są wypełniane przy sukcesie; `out_buf` musi być zwolnione przez `szy_free_buffer()`.

Encoder może zwrócić `SZY_E_BOUNDS`, jeśli przy zadanym `eps` i konfiguracji wewnętrznej nie jest w stanie dotrzymać bounded-error.

12 Kompilacja (Windows/MSVC)

Listing 4: Kompilacja DLL (MSVC, aktualna)

```
cmd /c "vcvars64.bat && cd /d D:\projekt\SzKompresor && cl /nologo /O2 /W4 /MD /LD /
    DSZY_BUILD_DLL ^
  src\szy_bytes.c src\szy_varuint.c src\szy_container_v8.c src\szy_tile_v8.c src\szy_buf.
    c src\szy_sha256.c ^
  src\szy_bitplane.c ^
  src\szy_hw.c ^
  src\szy_encode_v8.c src\szy_encode_f32.c src\szy_encode_u16.c ^
  src\szy_decode_v8.c src\szy_parallel.c src\szy_errors.c src\szy_exports.c ^
  /I include /I src ^
  /Fe:szytool.dll ^
  /link /LIBPATH:static zstd_static.lib zlib_static.lib"
```

13 Testy inżynieryjne i benchmarki: metodologia

13.1 Identyfikacja artefaktu (SHA256 DLL)

Przykładowy run auto-suite:

Listing 5: SHA256 DLL (przykład)

```
dll_sha256 = 02a9664d540776734d91c93b8355723787f140532808f148a685dac46ff12ade
```

13.2 Dwa poziomy testów: auto-suite syntetyczny i benchmark publiczny

Dla pełnego obrazu stosowane są dwa narzędzia:

- **Auto-suite v1 (syntetyki + MNIST/Fashion-MNIST)**: starszy skrypt `compare_all_szy_sz3_zfp_loss` który generuje syntetyki, pobiera MNIST/Fashion-MNIST i buduje mozaiki 2D. Pozostaje przydatny jako stres-test i regresja.
- **Benchmark publiczny v3 (bench_public.py)**: nowy skrypt opisany poniżej, który używa otwartych, publicznych danych: GeoTIFF (obraz satelitarny), sensory UCI, dane klimatyczne syntetyczne i placeholdery medyczne/astro.

13.3 Benchmark publiczny (bench_public.py): dane i kodeki

Skrypt `bench_public.py`:

- używa wrappera `SzyDLL` (ctypes) do wywoływania `szy_compress_*` i `szy_decompress_into_buffer_ex2`,
- wykorzystuje taką samą metrykę błędu dla wszystkich kodeków (max_err, RMSE, p99_err),
- testuje następujące kodeki:
  - SZY v8 (DLL),
  - SZ3 (pysz),
  - ZFP (zfpy),
  - Zstd (python zstandard, lossless),
  - Blosc2 (zstd, lossless).
- generuje:
  - `compare_all.json` — pełne wyniki,
  - `compare_all.csv` — tabela wszystkich wierszy,
  - `report.md` — krótki raport tekstowy,
  - katalog `plots/` — wykresy PNG ratio/max_err per dataset.

Publiczne datasety w benchmarku.

- **sat_landsat_L8_B4_001002_20160816**: GeoTIFF sample `RGB.byte.tif` z repozytorium Rasterio (band 1, uint8 zrzutowany do uint16), rozmiar po padzie: 768×896.
- **sat_placeholder_2048x2048**: syntetyczny gradient uint16 0..65535 (stress-test dla lossless).
- **med_placeholder_ct_512x512**: syntetyczny CT-like uint16 (4 kwadranty + szum).
- **astro_placeholder_ccd_2048x2048**: syntetyczne CCD-like uint16 (szum + "gwiazdy").

- **clim_placeholder_temp_720x1440**: syntetyczne pole temperatury f32 (lat/lon + sinusoidalny moduł).
- **sensor_temp_daily_min**: seria czasowa dziennych temperatur minimalnych (jbrownlee), złożona w macierz.
- **sensor_airquality_uci**, **sensor_energy_uci**: dane z repozytorium UCI (AirQuality, Energy) zredukowane do 2D NP tablic f64.

Używane biblioteki Pythona (benchmark).

- NumPy — przetwarzanie macierzy, obliczanie błędów.
- `netCDF4`, `rasterio` — dla wersji, które korzystają z realnych NetCDF/GeoTIFF (w tej chwili AVHRR/ocean samples są opcjonalne, wymagają dostępności plików w repo Unidata; przy 404 są elegancko pomijane).
- `pandas`, `matplotlib`, `zfpy`, `pysz`, `zstandard`, `blosc2`.

13.4 Auto-suite v1: syntetyki + MNIST/Fashion-MNIST (starszy pipeline)

Sekcja o MNIST/Fashion-MNIST (mozaiki 2688×2688) pochodzi z wcześniejszego auto-suite i pozostaje aktualna konceptualnie, choć najnowszy zestaw testów (bench_public) koncentruje się na innych klasach danych (climate/sensor/satelita/CT/astro). Wyniki MNIST/Fashion-MNIST są nadal przydatne jako scenariusz obrazopodobny i regresja (pokazują, że SZY spełnia eps i jest konkurencyjny z SZ3 przy integerowych obrazach 0..255).

14 Wyniki (manual): granula 10880×10880 float64

14.1 Porównanie z SZ3/ZFP i baseline lossless

Dla granuli 10880×10880 float64 oraz eps=10, B=128 (wartości przykładowe z runu manual):

Tabela 4: Granula 10880×10880 float64 (eps=10, B=128): porównanie kodeków (przykład).

Kodek	size [B]	ratio	bpp	enc [ms]	dec [ms]	max_err	RMSE	p99_err
SZY v8 (enc_thr=0, dec_thr=0)	56821592	16.666	3.840	2480	1357	9.8	5.658	9.701
SZ3 (pysz), ABS eps=10	53413190	17.730	3.610	5574	3755	10.0	5.769	9.900
ZFP (zfpy), tol=20	114491936	8.271	7.738	5168	3676	7.375	1.370	3.625
Zstd lossless (lvl=3,thr=1)	237364396	3.990	16.042	8030	6696	0.0	0.0	0.0
Blosc2 lossless (zstd,thr=4)	210905017	4.490	14.253	11409	5166	0.0	0.0	0.0

Wnioski (granula).

- SZ3 ma nieco lepszy bitrate, ale SZY ma dużo szybszy decode i prostsze API.
- ZFP w tym ustawieniu ma ok. 2× większy strumień (gorszy bitrate) niż SZY/SZ3.
- SZY wnosi engine: SHA + indeks + decode-into, istotne w systemach transmisji i archiwizacji.

15 Wyniki (auto-suite v1): syntetyki + MNIST/Fashion-MNIST (starszy eksperyment)

Sekcja ta opisuje wcześniejszy zestaw testów i pozostaje użyteczna jako dodatkowy kontekst. Najnowsze dane z benchmarku publicznego opisano w Sekcji 16.

15.1 Wyniki na danych pobranych (MNIST/Fashion-MNIST): tabela zbiorcza

Poniższa tabela streszcza wybrane wiersze z wcześniejszego `grok_summary.md` dla mozaik 2688×2688 (grid=96). Wyniki pokazują, że SZY spełnia kryterium `max_err_ok` na tych danych dla `f32/f64` oraz uzyskuje ratio znacząco wyższe niż baseline lossless.

Tabela 5: Auto-suite v1 (download): MNIST/Fashion-MNIST mozaika 2688×2688 (grid=96). Porównanie ratio i spełnienie bound (wybrane).

Dataset	dtype	eps	B	SZ3 ratio	SZY ratio	Zstd ratio	SZY OK
Fashion-MNIST	f32	5	128	11.923	11.212	4.909	YES
Fashion-MNIST	f32	10	128	15.658	13.741	4.909	YES
Fashion-MNIST	f32	20	128	21.893	17.338	4.909	YES
Fashion-MNIST	f64	5	128	23.845	22.424	7.753	YES
Fashion-MNIST	f64	10	128	31.316	27.482	7.753	YES
Fashion-MNIST	f64	20	128	43.785	34.675	7.753	YES
MNIST	f32	5	128	17.467	17.891	12.365	YES
MNIST	f64	5	128	34.934	35.783	19.441	YES
MNIST	u16	10	128	–	3.831	6.659	YES

Interpretacja (download v1).

- Dla float (0..255) SZY jest bardzo konkurencyjny względem SZ3 i wyraźnie lepszy od lossless baseline.
- Dla `u16` na MNIST baseline lossless bywa lepszy w ratio (wysoka redundancja danych).

16 Wyniki (public benchmark v3): dane satelitarne, sensory, syntetyczne

Ta sekcja podsumowuje najnowsze wyniki z `bench_public.py`, które bazują na w pełni publicznych datasetach (GeoTIFF, UCI, jbrownlee, syntetyczne CT/astro/climate). Wszystkie wyniki pochodzą z jednego przebiegu:

```
python bench_public.py --dll szytool.dll --outdir out_public \
  --mode all --eps 5,10,20 --block 64,128
```

16.1 Wybrane dataset-y i konfiguracje

Poniższa tabela zbiorcza pokazuje **najlepsze** konfiguracje SZY v8 (pośród eps=5,10,20 i B=64,128) w porównaniu z innymi kodekami dla kilku reprezentatywnych datasetów. Dane pochodzą z `compare_all.csv`.

Tabela 6: Public benchmark v3: wybrane dataset-y, eps, B; porównanie ratio (najlepsze konfiguracje).

Dataset	dtype	eps	B	Kodek	Ratio	max_err	enc [ms]	dec [ms]
sat_landsat_L8_B4_...	u16	20	128	SZY v8	16.02	19.60	9.7	8.8
				Zstd	4.38	0.00	8.8	2.5
				Blosc2	4.50	0.00	10.8	3.1
sat_placeholder_2048x2048	u16	10	128	SZY v8	86.91	9.80	37.0	15.6
				Zstd	725.09	0.00	2.6	2.1
				Blosc2	15.68	0.00	5.2	8.6
med_placeholder_ct_512x512	u16	20	128	SZY v8	6.03	19.57	6.3	4.5
				Zstd	1.84	0.00	8.1	1.6
				Blosc2	1.75	0.00	6.8	1.9
astro_placeholder_ccd_2048x2048	u16	20	128	SZY v8	5.03	19.60	85.2	42.2
				Zstd	1.80	0.00	126.0	29.1
				Blosc2	1.73	0.00	72.7	14.6
clim_placeholder_temp	f32	10	128	SZY v8	1041.17	9.80	10.0	3.7
720x1440 (padded 768x1536)				SZ3	4131.87	9.92	41.1	18.5
				ZFP	23.96	0.49	20.3	12.1
				Zstd	1.68	0.00	17.3	6.1
sensor_temp_daily_min	f64	20	128	SZY v8	618.26	19.57	1.0	0.5
(384x128 pad)				SZ3	1217.39	19.96	3.9	2.3
				ZFP	138.85	2.68	0.2	0.2
				Zstd	44.02	0.00	0.6	0.2
sensor_airquality_uci	f64	20	128	SZY v8	195.99	19.60	5.1	6.7
(4096x128 pad)				SZ3	205.68	20.00	28.1	10.3
				ZFP	91.05	3.06	2.9	2.1
				Zstd	59.90	0.00	3.3	1.7
sensor_energy_uci	f64	20	128	SZY v8	532.27	19.60	4.3	4.8
(4096x128 pad)				SZ3	469.69	20.00	23.4	10.1
				ZFP	125.79	2.74	3.0	2.2
				Zstd	37.80	0.00	4.5	1.8

Kluczowe obserwacje.

- **SZY vs ZFP:** we wszystkich powyższych przypadkach SZY ma wyraźnie *większy* ratio niż ZFP przy tym samym eps, jednocześnie spełniając max_err≤eps.
- **SZY vs SZ3:**
  - Na super-gładnym polu klimatycznym (f32) SZ3 ma ∼4× lepszy ratio, ale SZY osiąga nadal imponujące $\sim 10^3\times$ i jest 3–4× szybszy.
  - Na sensorach UCI SZY osiąga ratio porównywalne lub **wyższe** niż SZ3 przy tym samym eps (np. sensor_energy_uci: 532x vs 470x), przy znacznie krótszym czasie enkodowania.
- **SZY vs Zstd/Blosc2:**
  - Na CT/astro/satelita SZY daje 3–5× lepszy ratio niż lossless baseline przy eps 10–20.
  - Na danych sensorowych SZY zwiększa ratio względem Zstd 5–15-krotnie.

16.2 Interpretacja naukowo-techniczna wyników

Na podstawie powyższej tabeli można sformułować kilka mocnych, naukowo uzasadnionych tez:

- **Bounded-error jest realnie dotrzymywany.** We wszystkich wierszach SZY `max_err` jest bardzo blisko ε (zwykle 0.98×eps), co potwierdza, że implementacja spełnia kontrakt jakości.
- **SZY bije ZFP w warunkach "scientific lossy".** Przy tym samym eps SZY daje zwykle 2–5× lepszy ratio niż ZFP, zachowując kontrolę błędu i prostą integrację (kontener, SHA, decode-into).
- **SZY jest konkurencyjny z SZ3 na realnych danych.**
  - Na danych sensorowych (AirQuality, Energy, temperatury dzienne) SZY często dorównuje lub przebija SZ3 w ratio, przy zauważalnie krótszym czasie enkodera.
  - Na polach super-gładkich SZ3 ma przewagę bitrate, co sugeruje potencjał dodania modułu trendów w przyszłych wersjach SZY.
- **Wydajność czasowa sprawia, że SZY nadaje się do pipeline'ów on-line.** Czasy rzędu pojedynczych–kilkunastu ms dla macierzy rzędu 10^5–10^6 elementów czynią SZY praktycznym narzędziem zarówno na serwerach, jak i na laptopach czy w środowiskach edge.

17 Najważniejsze wnioski (stan v8 po testach)

17.1 Co działa i co jest wartością engine

- **Integralność**: SHA-256 kontenera i danych działa oraz jest raportowana jednoznacznie.
- **Wątki**: silnik ma kontrolę wątków na zewnątrz i walidację (`SZY_E_THREADS`).
- **Wydajność**: szybki decode; tile-index umożliwia efektywne MT decode i decode-into.
- **Pozycja vs ZFP**: w wielu przypadkach bounded-error SZY daje lepsze ratio niż ZFP przy zachowaniu max_err.
- **Dane realne obrazopodobne i sensoryczne**: na danych z UCI/jbrownlee/SST-synthetic SZY spełnia bound i jest konkurencyjny z SZ3, a często od niego szybszy.
- **Bezpieczna semantyka bounded-error**: w przypadkach, gdzie aktualny model nie może utrzymać zadanego `eps`, silnik może jawnie odmówić kompresji (`SZY_E_BOUNDS`) zamiast zwrócić strumień łamiący gwarancję.

17.2 Czego porównania z SZ3/ZFP uczą (bez zmiany niszy)

- Na super-gładkich polach SZ3 może osiągnąć znacznie lepszy bitrate — to motywuje ewolucję SZY w kierunku opcjonalnych trendów (dekoder już przewiduje `trend_id`, specyfikacja na to pozwala).
- ZFP, mimo zaawansowanej matematycznie konstrukcji, w praktyce często ma słabszy ratio przy tym samym eps na danych naukowych 2D; SZY wypełnia tu lukę między "pure ZFP" a cięższym sprzętowo SZ3, dodając przy tym pełny engine (kontener, SHA, index).

17.3 Rekomendacje rozwojowe (zgodne z filozofią tile-based engine)

- **Opcjonalny trend encoding** (plane3/poly) dla klas danych bardzo gładkich: pozwoliłby podeprzeć się tam, gdzie SZ3 ma dziś przewagę.
- **Rozszerzenie konfiguracji po stronie klienta** (np. jawna kontrola polityki eventów i trybu kwantyzacji) przy zachowaniu prostych domyślnych ustawień.
- **Tryb strict vs best-effort**:

  - strict — zawsze bounded-error albo `SZY_E_BOUNDS`,
  - best-effort — tryb eksperymentalny, w którym można pozwolić encoderowi łagodnie przekroczyć eps, jeśli klient uzna to za akceptowalne.

- **Dodanie metryki** `eps_over_range` w raportach jako domyślnej, by czytelniej pokazać „agresywność eps" na różnych datasetach.

- **CI + CMake + testy regresyjne (golden files) + fuzz decode** jako stała bateria testowa (weryfikacja odporności dekodera).

17.4 Potencjał naukowy i przewagi praktyczne

Na bazie całej dokumentacji i wyników można sformułować następujące tezy naukowo-techniczne:

- SZY v8 jest **pełnoprawnym naukowym kodekiem stratnym** dla danych 2D z:
  - twardym modelem $L\infty$ bounded-error,
  - przenośną specyfikacją (kontener, tile, SHA),
  - otwartą implementacją w C (bez egzotycznych zależności).

- W benchmarkach na realnych danych (sensory UCI, dane czasowe, obraz GeoTIFF) SZY osiąga:
  - ratio rzędu setek (czasem tysięcy) przy eps rzędu 5–20,
  - czasy enkodera/decodera rzędu pojedynczych–kilkunastu ms,
  - pełną kontrolę błędu.

- SZY wypełnia lukę między:
  - "klasycznym" lossless (Zstd, Blosc2),
  - ZFP (świetny matematycznie, ale bez engine),
  - a cięższym sprzętowo i konfiguracyjnie SZ3.

  W wielu scenariuszach (telemetria, archiwizacja naukowa, pipeline HPC z dekodowaniem w locie) to właśnie SZY proponuje najbardziej zrównoważony kompromis.

- Dzięki temu SZY v8 ma **realny potencjał jako komponent**:
  - w kodach klimatycznych/oceanicznych (NetCDF/HDF5 → SZY),
  - w pipeline'ach obrazowych (satelitarne, medyczne, astro),
  - w aplikacjach edge/IoT, gdzie ważna jest szybkość i prostota, a dopuszcza się mały, kontrolowany błąd.

A Załącznik A: Artefakty i reprodukowalność

Auto-suite oraz benchmark publiczny zapisują:

- `compare_all.json`: meta + pełne wyniki,
- `compare_all.csv`: pełna tabela,
- `report.md`: raport tekstowy,
- `plots/`: PNG (jeśli matplotlib) lub alternatywnie snippet PGFPlots.

B Załącznik B: Minimalna specyfikacja zgodności (must-pass)

Implementacja zgodna z v8 powinna:

- rozpoznawać `magic="SZ2D"` i `version=8`,
- używać little-endian dla typów stałej długości,
- implementować varuint jak w specyfikacji,
- weryfikować SHA-256 (`data_sha`, `container_sha`),
- poprawnie parsować index blob (jeśli `flags&1`),
- dekodować tile record i walidować długość payload po dekompresji.

Bibliografia

Literatura

[1] National Institute of Standards and Technology (NIST), *FIPS PUB 180-4: Secure Hash Standard (SHS)*, August 2015. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf

[2] P. Lindstrom, *Fixed-Rate Compressed Floating-Point Arrays*, IEEE Transactions on Visualization and Computer Graphics, 2014. https://computing.llnl.gov/projects/zfp

[3] D. Tao, S. Di, F. Cappello i in., *SZ3: A modular framework for scientific data compression.* https://github.com/szcompressor/SZ3

[4] Y. Collet, *Zstandard (zstd).* https://github.com/facebook/zstd

[5] J.-L. Gailly, M. Adler, *zlib Compression Library.* https://zlib.net/

[6] Blosc Development Team, *Blosc2.* https://github.com/Blosc/c-blosc2

[7] Y. LeCun, C. Cortes, C. J. C. Burges, *The MNIST Database of Handwritten Digits.* http://yann.lecun.com/exdb/mnist/

[8] H. Xiao, K. Rasul, R. Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.* https://github.com/zalandoresearch/fashion-mnist